Modeling

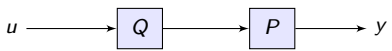
Unit 3

Negar Mehr

Models

- Modeling of physical systems is essential in modern engineering
- Models often consists of complex coupled differential equations, if then else logic, nonlinear functions, ... Solutions of these differential equations describe evolution of some physical system
- A model $\mathcal{M}$ has inputs u and outputs y, where u and y are signals

  - signals can be vector valued, i.e. many signals bundled together then model is called multi-input, multi-output
  - composition of models:



$$y = P(Q(u))$$

Uses of Models

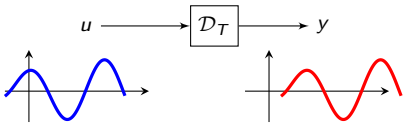
- Simulation
  - need a very detailed model
  - ex: virtual prototyping of a Boeing 777
  - simulations are run under 1000+ conditions (disturbances, noises, etc)
  - complex customized code: FEM, fluid dynamics, ...
- Prediction
  - ex: weather forecasting, solar power output for the next day, stock prices
  - uses historical data, generates forecast errors also
- Control and Design
  - need a simple model, because control complexity $\approx$ model complexity
  - iterative design process:
    modify design of aircraft wing, get new controller, simulate with detailed model, etc

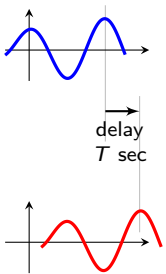
Modeling Errors

- Models
  - approximations of physical reality
  - no model is perfect: always have errors
- Modeling Errors: very important!
  - capture difference between model and physical reality
  - ex: could say model is $\pm 5\%$, accurate ... more complicated ways later
  - controller must be robust relative to plant modeling errors
- Range of models
  - very accurate, complex models: used in simulation and verification
  - very simple approximate models: used in control system design

The Delay Operator $\mathcal{D}_T$

- T-second delay operator: delays the input by $T > 0$ seconds
- symbol $\mathcal{D}_T$
- $y(t) = u(t - T)$





Properties of Models: Memoryless

- Model $\mathcal{M}$ is memoryless or static if $y(t)$ depends only on $u(t)$
  Otherwise, the output y at time t depends on input values at *other* times. In this case, the model $\mathcal{M}$ has memory, and is called dynamic
- Examples:
  - simple gain block: $y = Ku$ or $y(t) = Ku(t)$
  - memoryless nonlinearity: $y = \mathcal{N}(u)$ or $y(t) = \mathcal{N}(u(t))$
  - simple car model is not memoryless:

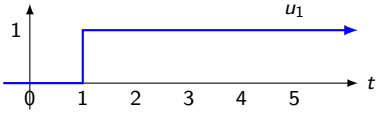
$$m\dot{y} + by = u$$

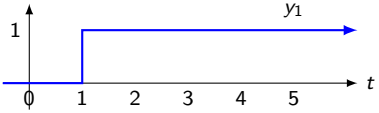
Causality

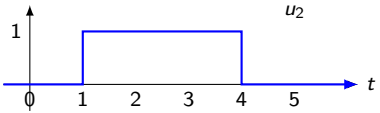
- Model $\mathcal{M}$ is called causal if $y(t)$ depends on $u(\tau)$ for $\tau \leq t$
- Model $\mathcal{M}$ is called strictly causal if $y(t)$ depends on $u(\tau)$ for $\tau < t$
  - think of t as current time, $\tau \leq t$ as the past, $\tau < t$ as the strict past
  - model is casual if output y at time t depends on past input values
  - model is strictly casual if output y at time t depends on strict past input values
- Of course all models of physical reality should be causal
  i.e. the input signals cause the output signals
  ex: throttle force on car causes change in car speed
- Causality is not entirely obvious in economic and social models:
  - does inflation cause unemployment, or is it the other way around?
  - does an increase in police officers cause an increase in crime rates, or is it the other way around?
  - determining causality from data is a big challenge in machine learning

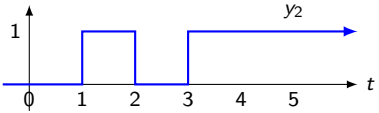
Example

- Ex: Consider model $\mathcal{M}$. Conduct two experiments: apply input u_1, observe output y_1 apply u_2, observe output y_2.
  - Observe that $u_1(t) = u_2(t) \quad \text{for } t < 4$
  - But $y_1(t) \neq y_2(t) \quad \text{for } t < 4$
- So this model is not causal!





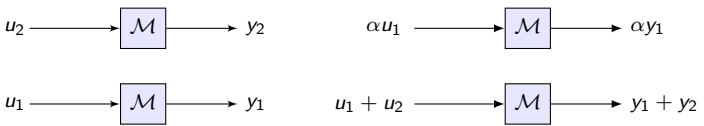




Properties of Models: Linearity

- Model $\mathcal{M}$ is called linear if for any input signals u_1 and u_2, and any real number α:

$$\begin{aligned} \mathcal{M}(\alpha u_1) &= \alpha \mathcal{M}(u_1) \quad (\textit{scaling}) \\ \mathcal{M}(u_1 + u_2) &= \mathcal{M}(u_1) + \mathcal{M}(u_2) \quad (\textit{additive}) \end{aligned}$$



- Otherwise, model is called nonlinear

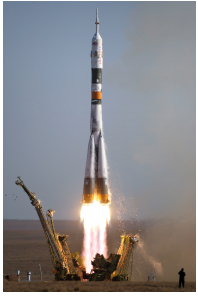
Properties of Models: Time-Invariance

- Model $\mathcal{M}$ is called time invariant if

$$\mathcal{M}D_T u = D_T \mathcal{M} u$$

for any input signal u and any delay T.

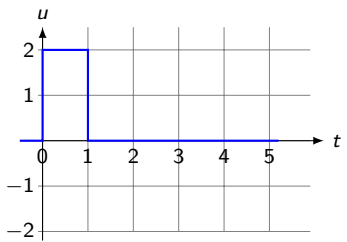
- Otherwise $\mathcal{M}$ is called time-varying
- We will most often deal with time-invariant models, but there are many practical systems that are time-varying
  - satellite in elliptical orbit
  - rocket that loses mass as it burns up fuel
  - chemical plants that have slag build up over time, decreasing their efficiency

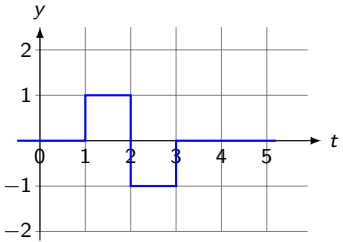


Pre-launch weight = 3MKg
fuel = 76.9
payload = 4.33%

Example

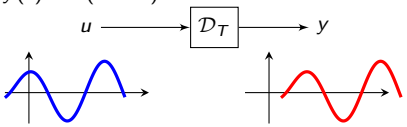
The input u shown below is applied to some LTI system $\mathcal{M}$. The resulting output y is plotted below. Assume that the initial conditions are zero. Sketch the unit step response of $\mathcal{M}$. Does you answer need causality?





Example

- Recall: T-second delay operator $\mathcal{D}_T$: delays the input by $T > 0$ seconds
- $y(t) = u(t - T)$



- is $\mathcal{D}_T$ linear? time-invariant? causal?

Cruise Control Car Model

y	speed of car
d	percent incline of roadway
m	mass of car

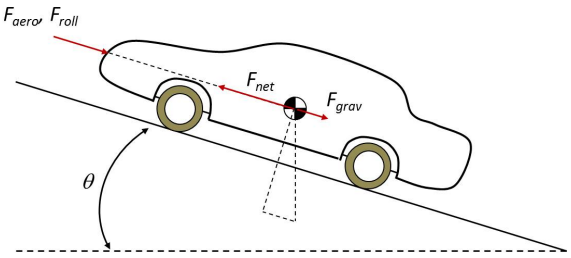
- for small inclines ($|\theta < 10^\circ$):
  incline of roadway $= \tan(\theta)$
  incline of roadway in percent $= 100\tan(\theta) = d$
- can approximate model using
  $\sin(\theta) \approx \tan(\theta) = d/100$

$$m\dot{y} \approx Eu - (mg/100)d - \alpha y$$

- even simpler model:
  ignore dynamics by setting $\dot{y} = 0$

$$y \approx \frac{E}{\alpha}u - \frac{mg}{100\alpha}d = Gu - Hd$$

$$m\dot{y} = \underbrace{Eu}_{\text{Engine thrust}} - \underbrace{mg\sin\theta}_{\text{roadway}} - \underbrace{\alpha y}_{\text{aero drag}}$$



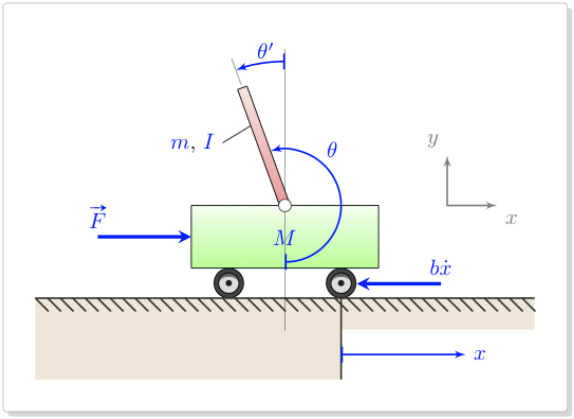
Inverted Pendulum

- similar to modeling bipedal robots
- very commonly used nonlinear example

$$M\ddot{x} + m(\ddot{x} + \frac{L}{2}cos\theta \cdot \ddot{\theta} - \frac{L}{2}sin\theta \cdot (\dot{\theta})^2) = F - b \cdot \dot{x}$$

$$\frac{L}{3}\ddot{\theta} + \frac{1}{2}\ddot{x} \cdot cos\theta = -\frac{1}{2}g \cdot sin\theta$$

- nonlinear, time-invariant

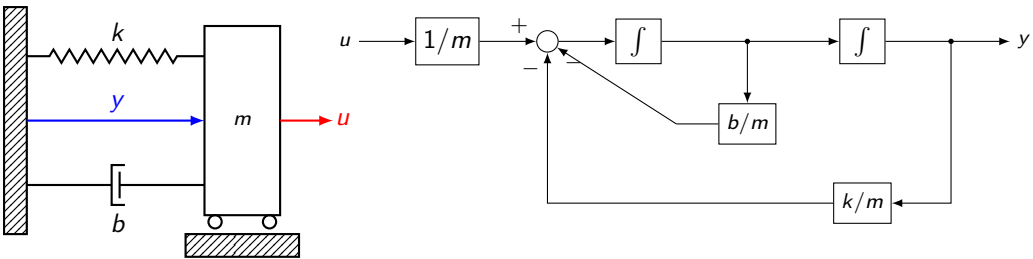


Spring-mass-damper

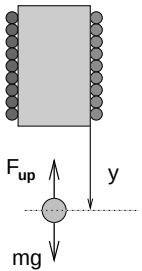
- can include saturation block to limit forces u
- useful to model car suspension systems

$$m\ddot{y} + b\dot{y} + by = u$$

- linear, time-invariant (if spring is linear)



Magnetically Suspended Ball

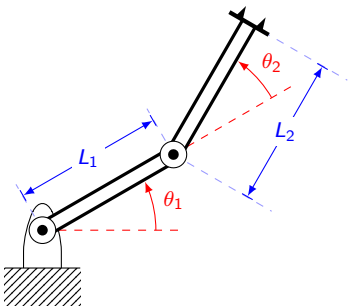


m	mass of ball
u	voltage applied to magnet coils
y	position of ball measured from base of magnet
c	magnetic coupling constant

Application: mag-lev train

$$m\ddot{y} = mg - \frac{cu^2}{y^2} \qquad \text{nonlinear, time-invariant}$$

Robot Manipulator



Equations of motion:

$$\begin{bmatrix} \alpha + 2\beta\cos(\theta_2) & \delta + \beta\cos(\theta_2) \\ \delta + \beta\cos(\theta_2) & \delta \end{bmatrix} \begin{bmatrix} \ddot{\theta}_1 \\ \ddot{\theta}_2 \end{bmatrix} + \begin{bmatrix} \beta\sin(\theta_2)\dot{\theta}_2 & -\beta\sin(\theta_2)(\dot{\theta}_1 + \dot{\theta}_2) \\ -\beta\sin(\theta_2)\dot{\theta}_1 & 0 \end{bmatrix} \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} = \begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix}$$

- α, β, δ are constants
- inputs τ_1, τ_2 are torques at the joints
- outputs are joint angles θ_1, θ_2
- model is nonlinear, time-invariant

Source: Murray, R., Li, Z., & Sastry, S. (1994). A mathematical introduction to robot manipulation. CRC Press.

General Input-Output Diff Eq Models

- General i/o diff eq model:

$$y^{[n]} = f\left(y^{[n-1]}, y^{[n-2]}, \cdots, \dot{y}, y, u^{[m]}, u^{[m-1]}, \cdots, \dot{u}, u, t\right)$$

- order $= n =$ highest derivative of output y
- If function f is nonlinear, model is nonlinear
  ex: $\ddot{y} + 3\dot{y}y + 7\cos(y) = 4u^2 + uy$
- If function f has an explicit time-dependence, model is time-varying
  ex: $\ddot{y} + 3t\dot{y}y + 7\cos(y) = 4u^2 + uyt^2$

General Input-Output Diff Eq Models ...

$$y^{[n]} = f\left(y^{[n-1]}, y^{[n-2]}, \cdots, \dot{y}, y, u^{[m]}, u^{[m-1]}, \cdots, \dot{u}, u, t\right)$$

- Function f is called linear if it involves linear combinations of $y^{[n-1]}, y^{[n-2]}, \cdots, \dot{y}, y, u^{[m]}, \cdots, \dot{u}, u$
  If function f is linear, model is linear
- General LTI diff eq i/o model

$$y^{[n]} + a_{n-1}y^{[n-1]} + \cdots + a_1\dot{y} + a_0 y = b_m u^{[m]} + b_{m-1}u^{[m-1]} + \cdots + b_1\dot{u} + b_o u$$

  ex: $\ddot{y} + 3\dot{y} + 7y = 4\dot{u} - 2u$
- relative degree $\delta = n - m$ = highest derivative of output y - highest derivative of input u
- if $n \geq m$ model is causal, if $n > m$ model is strictly causal,
  i.e. relative degree $\delta \geq 0$ for causality, and $\delta > 0$ for strict causality

Causality Explanation

- Why do we need $n \geq m$ for causality, and $n > m$ for strict causality?
- Consider first order

$$\dot{y} + ay = bu \quad \text{here } n = 1$$

  - approximate the derivative as a forward finite difference:

$$\dot{y}(t) \approx \frac{y(t+h) - y(t)}{h}$$

  - then diff eq becomes

$$\frac{y(t+h) - y(t)}{h} \approx -ay(t) + bu(t) \quad \implies y(t+h) \approx (1 - ah)y(t) + bhu(t)$$

  - clearly strictly causal as $y(t+h)$ depends only on strictly past values of u!
  - easy to generalize to higher order ODEs (linear and nonlinear)

Focus of this Class

- All our models will be causal
- Most of the class will focus on LTI diff eq models:

$$y^{(n)} + a_{n-1}y^{(n-1)} + \cdots + a_1\dot{y} + a_0 y = b_m u^{(m)} + b_{m-1}u^{(m-1)} + \cdots + b_1\dot{u} + b_o u$$

- Last unit of class will focus of nonlinear TI models:

$$y^{[n]} = f\left(y^{[n-1]}, y^{[n-2]}, \cdots, \dot{y}, y, u^{[m]}, u^{[m-1]}, \cdots, \dot{u}, u\right)$$

- Remark: all properties extend to MIMO models:
  just consider one input at a time

Finding Models from Data

- Previous examples: models constructed from basic physics
- Sometimes that is too hard, so we construct models from data
- Parameter estimation
  - might use physics to guide form of model, leaving unknown parameters
  - unknown parameters found from experimental data
- System Identification or black-box modeling
  - throw physics out the window
  - try a model form with unknown parameters
  - unknown parameters found from experimental data
- In both cases, we estimate some parameters from expt data

Norms

- Norms are a measure of length
  - Notation norm of v is written $\|v\|$
  - many different choices
- Vectors $\|v\| = \sqrt{v_1^2 + v_2^2 + \cdots v_n^2} = \sqrt{v^T v}$ generalizes Euclidean distance
  - ex: $\left\| \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}^T \right\| = \sqrt{1^2 + 2^2 + 3^2} = \sqrt{14}$
- Signals u defined on $[t_o, t_f]$
  - continuous-time signals: $\|u\| = \sqrt{\int_{t_o}^{t_f} u^2(t) dt}$
  - discrete time signals: basically the same as vectors! $\|v\| = \sqrt{v_1^2 + v_2^2 + \cdots v_n^2} = \sqrt{v^T v}$
- Distance between two vectors or two signals is $\|u - v\|$
- in Matlab `>> norm(v)`

Parameter Estimation: Example

- Car model $\mathcal{M}$ with unknown aero drag coefficient: $m\dot{y} = Eu - \alpha y$
- Want to estimate parameter α
- Algorithm:
  - experimental data: u^d, y^d
  - simulate model for various values of α with input u^d, i.e. compute $y = \mathcal{M}(u^d, \alpha)$
  - see which one is closest to expt observed output y^d, i.e. solve optimization problem:

$$\min_{\alpha} \|y^d - \mathcal{M}(u^d, \alpha)\| \quad \rightarrow \alpha^{\text{est}} = \text{best estimate of } \alpha$$

- What makes this algorithm slow:
  - simulation step takes time
  - searching over parameters, especially if there are many

Parameter Estimation : code

- Many simulation calls ...
- Not good to use Simulink, use command line simulation with ode45

```
alphavals = [0:0.1:1]';
L = length(alphavals);
cost = zeros(L,1);
for ii = 1:L
alphanow = alphavals(ii);
    [t, y] = ode45( );
    cost(ii) = norm(y - ydata);
end
[mincost, iibest] = min(cost);
alphabest = alphavals(iibest);
plot(alphavals, cost);
```

ode45

- Command line solver for differential equations: ex: simulate car model $m\dot{y} = Eu - \alpha y$
- Create this function

```
function ydot = car(t,y,utimes, udata,alpha)
% load constants
m = 1000; E = 720;
unow = interp1(utimes,udata,t);
ydot = (E/m)*unow - (alpha/m)*y;
```

- Run this script

```
utimes = [0:0.1:10];
udata = randn(size(times));
tspan = [1 10];
yinit = 35;
alpha = 0.6;
opts = odeset('RelTol',1e-2,'AbsTol',1e-4);
[t,y] = ode45(@(t,y) car(t,y,utimes,udata,alpha), tspan, yinit, opts);
```